Major Project Synopsis Report

CampusGenie AI

Project Category: University Based Project

Projexa Team Id - **2604043**

Team CYC - Control Your Career

Submitted in partial fulfilment of the requirement of the degree of

Master of Computer Applications

Section -A

to

K.R Mangalam University

by

Student Name : Bhanu Asati (2501560014) (Leader)

Student Name : Srashti Dwivedi (2501560012)

Under the supervision of

Mr. Vishwanil Suman

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

August 2026

INDEX

S. No		Page No.
0.	Abstract	
1.	Introduction	1
2.	Motivation	2
3.	Literature Review 4.1 Chatbots in Education 4.2 Retrieval-Augmented Generation for Knowledge-Based Question Answering 4.3 Reasoning and AI Agents 4.4 Chatbot Technology and Conversational Systems	3-5
4.	Gap Analysis 5.1 Research Gap 5.2 Identified Research Gap 5.3 Major Problems Identified 5.4 Proposed Problem Resolution	6-7
5.	Problem Statement	8-9
6.	Objectives 7.1 Specific Objectives	10
7.	Tools/platform Used 8.1 Reasons for Selecting the Programming Language 8.2 Reasons for Selecting Python	11-12
8.	Methodology 9.1 Document Collection and Pre-processing 9.2 Knowledge Base Creation 9.3 User Query Processing 9.4 Information Retrieval using RAG 9.5 Response Generation 9.6 Agentic AI Workflow 9.7 Web Application Integration 9.8 Testing and Evaluation	13-25
9.	References	26

ABSTRACT

CampusGenie is an AI-powered academic assistant designed to make university-related information easily accessible, accurate, and conversational for students and other academic stakeholders. In educational institutions, important information such as course details, academic schedules, notices, attendance policies, placement updates, examination guidelines, and university regulations is often distributed across multiple documents and platforms. Finding relevant information from these sources can be time-consuming and may lead to incomplete or inaccurate understanding.

CampusGenie addresses this problem by combining **Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and Agentic AI to create an intelligent knowledge-access platform.** The system retrieves relevant information from institutional documents and uses an AI model to generate context-aware responses to users' natural-language queries. The RAG architecture helps ground responses in available institutional knowledge, thereby reducing the possibility of generating unsupported information. An agent-based architecture can further enable the system to handle different categories of academic tasks through specialized intelligent workflows.

The proposed system is implemented as a web-based application using Python, React.js, and FastAPI, with LangChain and LangGraph for AI orchestration and workflow management. OpenAI or Gemini APIs are used as the underlying language model, while vector databases such as **ChromaDB or FAISS** are used for efficient semantic retrieval of relevant information.

CampusGenie aims to provide a scalable, user-friendly, and future-ready academic assistance platform. In addition to question answering, the system can be extended to support document summarization, assignment assistance, voice-based interaction, personalized recommendations, and other intelligent academic services. The project provides practical experience in developing modern AI applications and demonstrates how LLMs, RAG, Vector Databases, and Agentic AI can be integrated to solve real-world problems in the education domain.

Keywords: **Artificial Intelligence (AI), Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), Agentic AI, Conversational AI, Natural Language Processing (NLP), Vector Database, Semantic Search, Academic Assistant.**

1. INTRODUCTION

The rapid growth of **Artificial Intelligence (AI)** has transformed the way information is accessed, processed, and communicated. Technologies such as **Large Language Models (LLMs), Natural Language Processing (NLP), Retrieval-Augmented Generation (RAG), and Agentic AI** have enabled the development of intelligent applications that can understand natural-language queries and provide meaningful, context-aware responses. These advancements have created significant opportunities for applying AI in the education sector.

Universities and colleges generate a large amount of **academic and administrative information,** including course details, examination schedules, attendance requirements, notices, placement updates, and institutional policies. Although this information is available through websites, portals, PDFs, and circulars, students often need to search through multiple sources to find a specific answer. Traditional keyword-based search methods may also fail to understand the actual intent of a student's query, making information access time-consuming and inconvenient.

CampusGenie – AI for Smarter Learning is proposed as an AI-powered academic assistant that provides a conversational interface for accessing institutional knowledge. The system integrates LLMs, RAG, and Agentic AI to understand user queries, retrieve relevant information from institutional documents, and generate accurate and context-aware responses. The use of RAG helps reduce dependence on the model's pre-trained knowledge by providing relevant institutional content as context during response generation.

From a technical perspective, CampusGenie is designed as a modern web-based application using React.js, Python, FastAPI, LangChain, LangGraph, vector databases such as ChromaDB/FAISS, and OpenAI/Gemini APIs. The project aims to make academic information easier to access, faster to find, and simpler to understand, while providing a scalable foundation for future features such as document summarization, assignment assistance, voice interaction, and personalized academic support.

2. MOTIVATION

The motivation for developing CampusGenie comes from my own experience as a university student. During my academic journey, I have often needed information about class schedules, notices, attendance, examinations, placements, academic activities, and university-related procedures. In many cases, the required information was available, but finding it was difficult because it was shared through different platforms such as WhatsApp groups, university portals, notices, PDFs, and messages from faculty members. Sometimes I had to ask the same question to classmates or faculty members simply because I could not quickly find the official information.

Another issue I experienced was that even when a document or notice was available, it was not always easy to understand exactly what was relevant to my question. For example, instead of reading an entire notice or searching through a long PDF, it would be much more convenient to simply ask, “What is the last date for this registration?” or “What are the eligibility requirements?” and receive a direct answer based on the official information.

At the same time, the increasing use of tools such as ChatGPT and other AI assistants showed me how convenient conversational interaction can be. This raised the idea of combining that experience with university-specific information. The motivation behind CampusGenie is therefore to build an assistant that does not simply generate general AI responses, but can retrieve relevant institutional information and provide it in a simple, conversational manner.

Through this project, I also wanted to move beyond learning AI concepts theoretically and understand how technologies such as LLMs, RAG, vector databases, and AI agents can be used to solve a problem that students actually face. CampusGenie is therefore motivated by both a real student need and the opportunity to build a practical, scalable AI solution for the academic environment.

3. LITERATURE REVIEW

The development of intelligent academic assistants has been supported by research in chatbots, conversational AI, Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and AI agents. Existing studies show that conversational systems can improve access to educational information, while retrieval-based approaches can make language-model responses more relevant to specific knowledge domains.

CHATBOTS IN EDUCATION

Okonkwo and Ade-Ibijola (2021) conducted a systematic review of 53 studies on the application of chatbots in education. Their study found that chatbots have potential to provide quick and personalized services to students and educational staff. The review also identified challenges related to implementation, usability, and reliability. The research establishes that conversational systems can be useful in educational environments, but their effectiveness depends on how they are designed and evaluated. This provides a strong foundation for developing CampusGenie as a conversational academic assistant.

Wollny et al. (2021) performed a systematic literature review of 74 relevant publications after examining 2,678 publications related to chatbots in education. Their findings showed that educational chatbots have been used for learning support, assistance, mentoring, and administrative information. The authors also identified the need for better adaptation and evaluation of educational chatbots. This indicates that a useful academic assistant should not be limited to simple question answering, but should be designed around actual student needs.

RETRIEVAL-AUGMENTED GENERATION FOR KNOWLEDGE-BASED QUESTION ANSWERING

Lewis et al. (2020) introduced Retrieval-Augmented Generation (RAG), combining a language model with an external non-parametric knowledge source. Their experiments showed that retrieval-augmented models can produce more specific and factual responses than models relying only on parametric knowledge. The study is particularly relevant to CampusGenie because university information is institution-specific and can change over time. Using retrieval before generation allows the system to ground its responses in available institutional documents.

Gao et al. (2023) presented a comprehensive survey of RAG for Large Language Models. The study explains that LLMs can face problems such as outdated knowledge, hallucination, and difficulty incorporating domain-specific information. The authors describe retrieval, augmentation, and generation as the major components of RAG systems and discuss their use for continuously updated and domain-specific knowledge. These findings support the use of a vector database and retrieval pipeline in CampusGenie for accessing university-specific information.

REASONING AND AI AGENTS

Yao et al. (2022) introduced ReAct (Reasoning and Acting), a framework that combines reasoning with actions that allow a language model to interact with external sources. Their experiments demonstrated that combining reasoning and external actions can improve performance on question answering and decision-making tasks while providing more interpretable task-solving behaviour. This research is relevant to the Agentic AI component of CampusGenie, where different stages such as intent understanding, retrieval, and response generation can be coordinated as part of an intelligent workflow.

CHATBOT TECHNOLOGY AND CONVERSATIONAL SYSTEMS

Adamopoulou and Moussiades (2020) reviewed the history, technologies, and applications of chatbot systems. Their work describes the transition from traditional pattern-matching systems toward machine-learning-based conversational systems and highlights important design considerations, applications, and limitations. This research provides the broader technical background for designing the conversational interface of CampusGenie.

RESEARCH GAP

The reviewed studies demonstrate that educational chatbots can improve information access, RAG can improve knowledge-grounded responses, and agent-based approaches can allow language models to interact with external resources and perform multi-step tasks. However, these areas are often studied separately. CampusGenie brings these ideas together into a single academic assistant focused on institution-specific university information. The proposed system therefore aims to combine conversational interaction, document retrieval, LLM-based generation, and agent workflows into one practical platform for students.

Author / Year	Sample/ Focus	Title / Study	Source	Major Findings / Relevance to CampusGenie
Adamopoulou & Moussiades (2020)	Review of chatbot technologies and applications	Chatbots: History, Technology, and Applications	Machine Learning with Applications	Discusses the evolution of chatbots from pattern matching to machine-learning approaches and identifies important design considerations for conversational systems. (ScienceDirect)
Lewis et al. (2020)	Knowledge-intensive NLP and question answering	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	NeurIPS 2020	Introduced RAG by combining parametric language models with external knowledge retrieval; showed improved factual and specific generation. (ML Anthology)
Okonkwo & Ade-Ibijola (2021)	Systematic review of 53 studies	Chatbots Applications in Education: A Systematic Review	Computers and Education: Artificial Intelligence	Found that chatbots can provide quick and personalized educational services while identifying challenges and future research areas. (University of Johannesburg)
Wollny et al. (2021)	74 relevant publications from 2,678 screened	Are We There Yet? – A Systematic Literature Review on Chatbots in Education	Frontiers in Artificial Intelligence	Identified applications in learning, assistance, mentoring, and administrative support; highlighted adaptation and evaluation as major research challenges. (Frontiers)
Yao et al. (2022)	Reasoning, QA and interactive decision-making tasks	ReAct: Synergizing Reasoning and Acting in Language Models	arXiv / Google Research	Demonstrated that combining reasoning and actions enables LLMs to interact with external knowledge and supports more effective task solving. (arXiv)
Gao et al. (2023)	Survey of RAG methods and architectures	Retrieval-Augmented Generation for Large Language Models: A Survey	arXiv	Identified outdated knowledge, hallucination, and domain-specific knowledge as key LLM limitations and presented RAG as a solution using retrieval, augmentation, and generation. (arXiv)

4. GAP ANALYSIS

The literature review shows that significant work has been done in the areas of educational chatbots, Large Language Models, Retrieval-Augmented Generation (RAG), and AI agents. However, most existing systems focus on one particular capability, such as general student assistance, chatbot-based learning, or knowledge retrieval. There is a need for a unified system that combines these technologies specifically for accessing institution-specific academic information.

The major gaps identified from the existing studies are:

Area	**Existing Approach**	**Identified Gap**	**CampusGenie Approach**
Information Access	Students search websites, portals, PDFs, and notices separately.	Information is scattered across multiple sources.	Provides a single conversational interface for institutional information.
Chatbot Systems	Many educational chatbots provide predefined or general responses.	Limited understanding of institution-specific queries.	Uses LLMs with institutional knowledge for contextual responses.
Knowledge Accuracy	General LLMs depend mainly on pre-trained knowledge.	Responses may be outdated or unsupported.	Uses RAG to retrieve relevant information before generation.
Semantic Retrieval	Traditional systems often rely on keyword matching.	Keyword search may fail to understand query meaning.	Uses embeddings and vector search for semantic retrieval.
Agentic Capabilities	Many systems are limited to question answering.	Limited support for multi-step academic tasks.	Introduces AI-agent workflows for intent detection, retrieval, and response handling.
Academic Personalization	Existing systems often provide the same type of response to different users.	Limited scope for personalized academic assistance.	Provides a foundation for personalized and context-aware services.
Future Extensibility	Individual systems are often built for a fixed purpose.	Difficult to extend to new academic services.	Designed as a modular and scalable architecture for future features.

Identified Research Gap

The key gap identified is the lack of a single, knowledge-grounded, conversational academic platform that combines LLMs, RAG, vector-based semantic search, and Agentic AI to work with institution-specific information. Existing research demonstrates the effectiveness of these technologies individually, but CampusGenie focuses on integrating them into one practical system for students.

Therefore, CampusGenie attempts to bridge the gap between traditional academic information systems and modern intelligent assistants by providing a platform where students can ask questions naturally and receive responses based on relevant institutional knowledge. This approach can make academic information more accessible, efficient, and easier to understand, while also providing a foundation for future intelligent university services.

5. PROBLEM STATEMENT

In a university environment, students frequently require information related to courses, examinations, attendance, schedules, notices, placements, and academic policies. Although this information is available through different platforms, students often face difficulty in locating the exact information they need.

Major Problems Identified:

- **Scattered Information:** Academic information is distributed across websites, portals, PDFs, notices, WhatsApp groups, and other communication channels.
- **Time-Consuming Search:** Students often spend considerable time searching through multiple sources for a simple query.
- **Keyword-Based Search:** Traditional search systems may not understand the actual intent and context of a student's question.
- **Dependence on Others:** Students may need to contact faculty members, administrative staff, or classmates for information that is already available.
- **Lack of Context-Aware Responses:** General-purpose AI tools may provide answers that are not specific to the university or its current policies.
- **Outdated or Unsupported Information:** Without access to institutional documents, an AI system may generate information that is incomplete, outdated, or not supported by official sources.
- **Limited Intelligent Assistance:** Existing systems generally focus on information display or search rather than providing a conversational and intelligent academic experience.

Proposed Problem Resolution:

To address these challenges, **CampusGenie** is proposed as a **centralized AI-powered academic assistant** that combines **Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), semantic search, vector databases, and Agentic AI.**

The system aims to:

- Provide a single conversational interface for academic information.
- Understand queries expressed in natural language.
- Retrieve relevant information from institutional documents.
- Generate context-aware and knowledge-grounded responses.
- Provide a scalable foundation for future features such as document summarization, assignment assistance, voice interaction, and personalized academic support.

Thus, the problem addressed by CampusGenie is the difficulty of accessing and understanding scattered university information efficiently and intelligently, and the project aims to bridge this gap through a modern AI-based solution.

6. OBJECTIVES

The primary objective of CampusGenie – AI for Smarter Learning is to develop an intelligent academic assistant that simplifies access to university-related information through natural-language interaction. The project aims to combine modern AI technologies with institutional knowledge to provide useful, relevant, and context-aware academic assistance.

<u>Specific Objectives:</u>

I. Develop an AI-powered academic assistant capable of understanding and responding to students' natural-language queries.

II. Implement a Retrieval-Augmented Generation (RAG) system to retrieve relevant information from institutional documents before generating responses.

III. Create a semantic knowledge retrieval system using embeddings and a vector database such as ChromaDB or FAISS.

IV. Integrate Large Language Models (LLMs) such as OpenAI or Gemini to generate meaningful and conversational responses.

V. Implement Agentic AI workflows for tasks such as query understanding, intent detection, information retrieval, and response generation.

VI. Provide a centralized web-based interface through which students can easily access academic and institutional information.

VII. Reduce the time and effort required to find university information from multiple websites, documents, and communication channels.

VIII. Improve the relevance and reliability of responses by grounding generated answers in available institutional knowledge.

IX. Design a scalable architecture that can be extended with features such as document summarization, assignment assistance, voice interaction, and personalized recommendations.

X. Gain practical experience in developing and integrating LLMs, RAG, vector databases, AI agents, APIs, and full-stack web technologies into a real-world application.

7. Tools/Technologies Used

CampusGenie uses a combination of **web development, Artificial Intelligence, Retrieval-Augmented Generation, and database technologies** to build a complete academic assistance platform.

Technology / Tool	***Purpose / Usage in CampusGenie***
Python	Core programming language for backend development and AI implementation.
React.js	Used to develop the interactive and responsive frontend of the application.
FastAPI	Provides backend APIs and connects the frontend with AI and database services.
LangChain	Used for LLM integration, document processing, retrieval pipelines, and AI application development.
LangGraph	Used to design and manage agent-based workflows and multi-step AI processes.
OpenAI API / Gemini API	Provides the Large Language Model used for understanding queries and generating responses.
RAG (Retrieval-Augmented Generation)	Retrieves relevant institutional information before generating an answer.
ChromaDB / FAISS	Stores vector embeddings and performs semantic similarity search.
Embedding Model	Converts documents and user queries into numerical vector representations for semantic retrieval.
HTML & CSS	Used for structuring and styling web interface components where required.
JavaScript	Supports frontend interaction and client-side functionality.
Git & GitHub	Used for source-code management, version control, and collaborative development.
VS Code	Primary development environment for writing, debugging, and managing the project.
Postman	Used for testing and validating backend REST APIs.
PyPDF / Document Loaders	Used for extracting and processing content from institutional PDF/document sources.

Technology Architecture :

React.js
↓
FastAPI / Python Backend
↓
LangChain + LangGraph
↓
RAG Pipeline + AI Agents
↓
ChromaDB / FAISS
↓
OpenAI / Gemini LLM

Reasons for Selecting the Programming Language :

Python has been selected as the primary programming language for developing CampusGenie because the project mainly involves Artificial Intelligence, Large Language Models, RAG pipelines, document processing, and backend API development. Python provides a strong ecosystem of libraries and frameworks required for implementing these components efficiently.

Reasons for Selecting Python :

- Strong AI/ML Ecosystem: Python provides extensive libraries and frameworks for AI and machine learning, making it suitable for LLM-based application development.
- LLM Integration: APIs and SDKs for models such as OpenAI and Gemini can be integrated easily using Python.
- RAG Development: Python has well-supported tools such as LangChain, LangGraph, embedding libraries, and vector-database integrations, which simplify the development of RAG pipelines.
- Easy Document Processing: Python provides libraries for processing PDFs, text files, and other academic documents, which are important for building the institutional knowledge base.
- FastAPI Support: Python supports FastAPI, allowing the project to expose fast and lightweight REST APIs between the frontend and AI backend.
- Large Community Support: Python has a large developer community and extensive documentation, making troubleshooting and implementation easier.
- Simple and Maintainable Syntax: Python's readable syntax helps reduce development complexity and makes the code easier to maintain and extend.

8. METHODOLOGY

The methodology of CampusGenie – AI for Smarter Learning is designed to provide students with relevant academic information through a conversational AI interface. The system follows a Retrieval-Augmented Generation (RAG) based approach in which institutional documents are processed, stored as searchable knowledge, and retrieved according to the user's query before generating the final response.

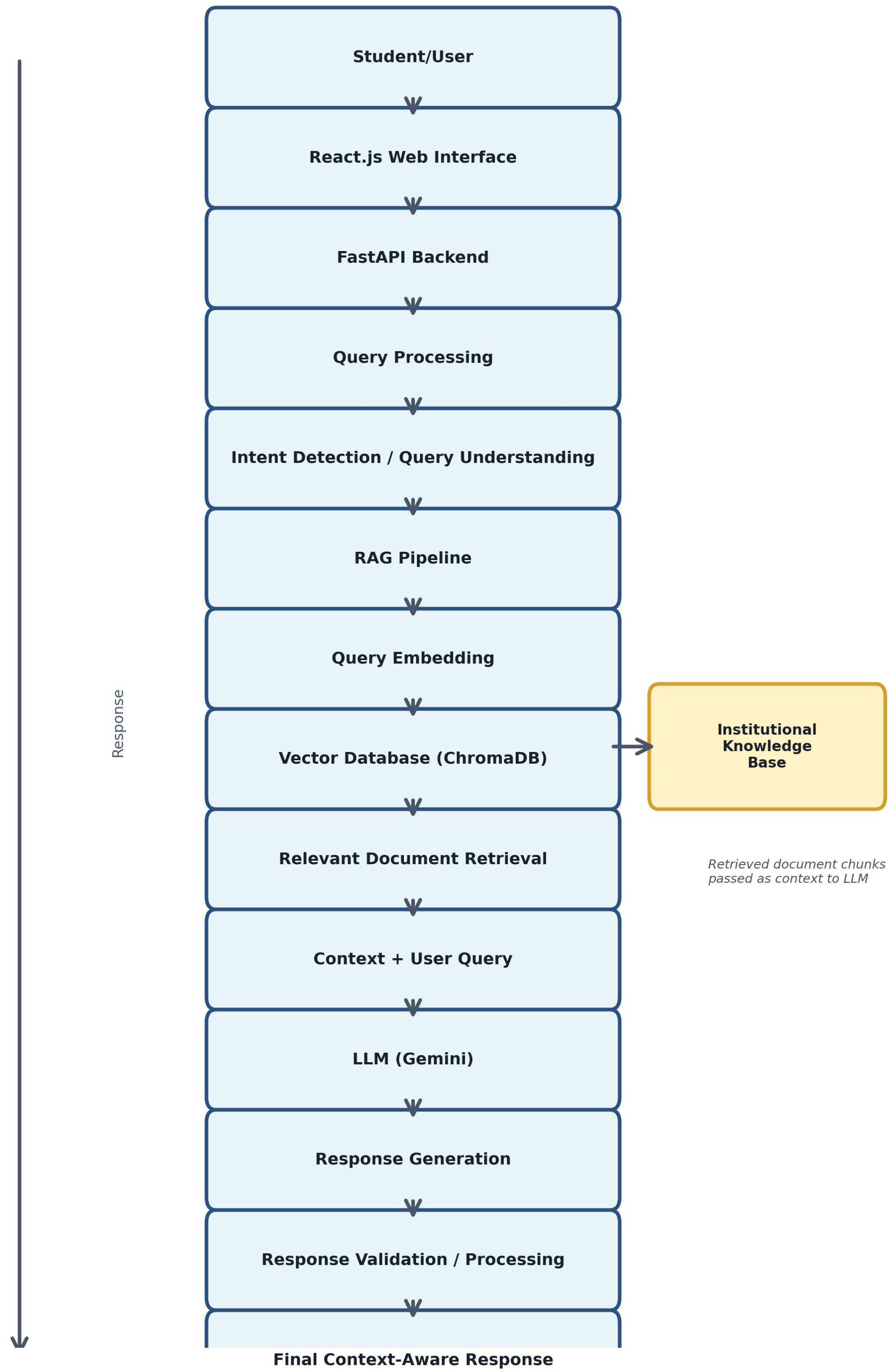
8.1 Document Collection and Pre-processing

Relevant academic and institutional documents such as university notices, course details, examination guidelines, attendance policies, placement information, and academic regulations are collected as the primary knowledge source.

The collected documents are cleaned and converted into text. Large documents are divided into smaller meaningful chunks so that relevant information can be retrieved efficiently.

Documents → Text Extraction → Cleaning → Chunking

Figure 8.1: Overall Methodology of CampusGenie



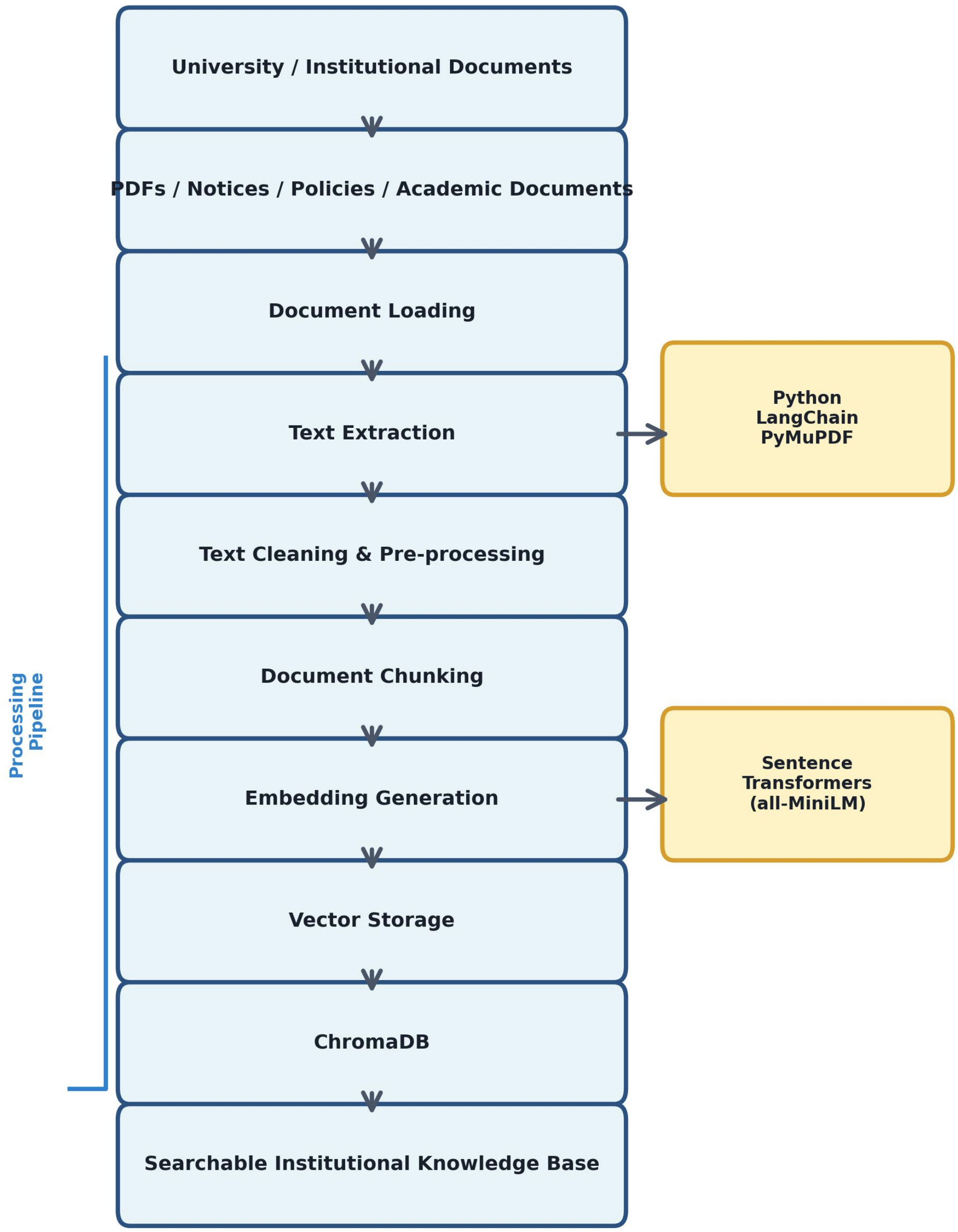
8.2 Knowledge Base Creation

After preprocessing, each document chunk is converted into a numerical representation called an embedding. These embeddings represent the semantic meaning of the content and are stored in a vector database such as ChromaDB or FAISS.

This creates a searchable institutional knowledge base that allows the system to find information based on meaning rather than only exact keywords.

Text Chunks → Embeddings → Vector Database

Figure 8.2: Institutional Knowledge Preparation Pipeline



This knowledge base is used by the RAG pipeline for context retrieval

8.3 User Query Processing

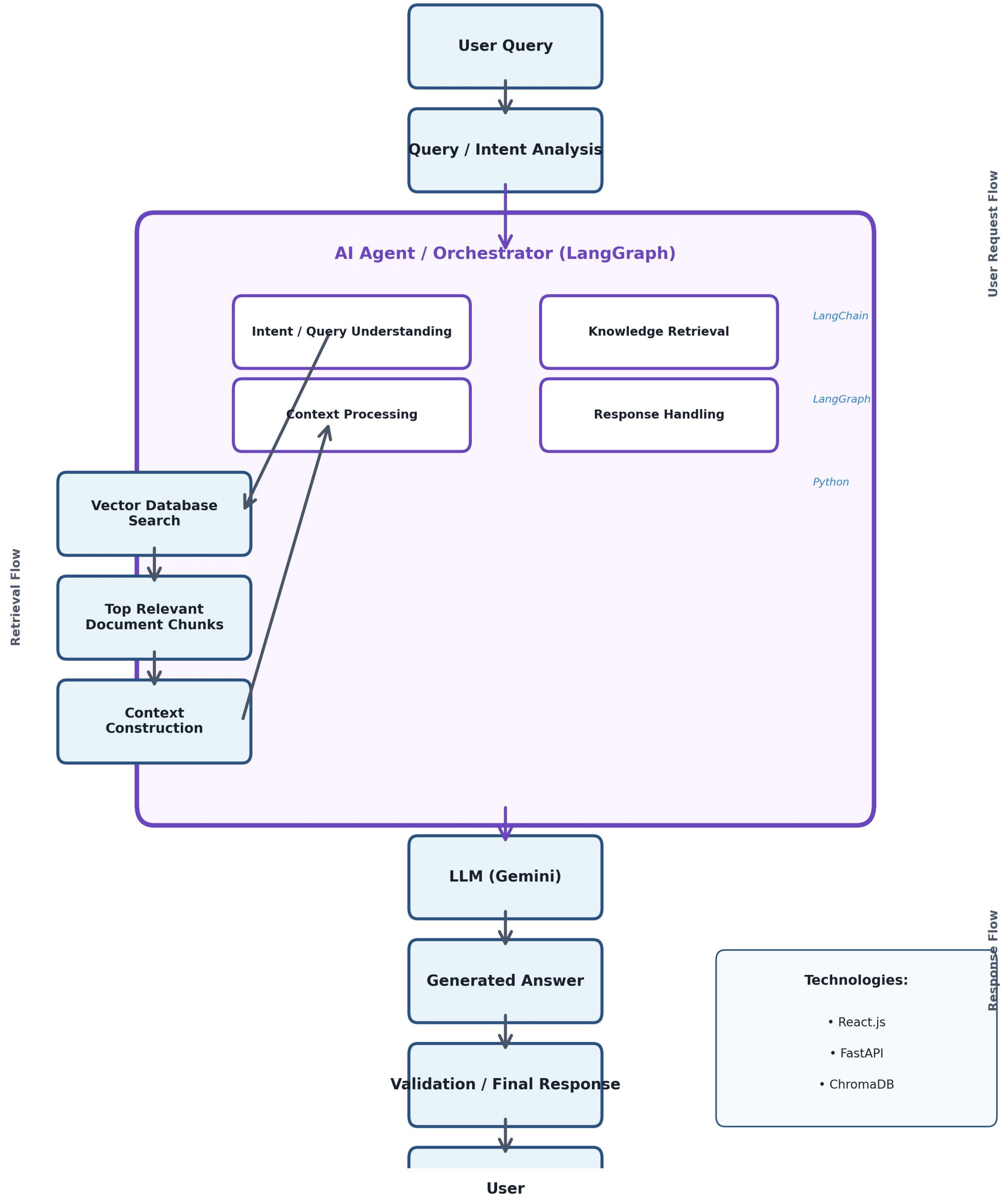
When a student enters a question through the web application, the query is first processed by the backend. The system identifies the meaning of the query and converts it into an embedding.

For example:

“What is the minimum attendance required for the examination?”

Instead of searching only for the exact words, the system looks for documents containing semantically related information about attendance and examination requirements.

Figure 8.3: RAG and Agentic AI Workflow



8.4 Information Retrieval using RAG

The query embedding is compared with the embeddings stored in the vector database. The system retrieves the most relevant document chunks and uses them as context for the language model.

The retrieval process can be represented as:

User Query → Query Embedding → Semantic Search → Relevant Document Chunks

This approach allows CampusGenie to use institution-specific and document-based information while generating responses.

8.5 Response Generation

The retrieved context is combined with the user's original question and passed to the selected Large Language Model (LLM) such as OpenAI or Gemini.

The LLM then generates a natural-language response based on the retrieved information.

User Query + Retrieved Context → LLM → Final Response

The objective is to provide answers that are relevant, understandable, and grounded in the available institutional knowledge.

8.6 Agentic AI Workflow

To make the system more flexible, Agentic AI can be used to coordinate different stages of the workflow. Depending on the query, different components can perform specialized tasks such as:

Intent Detection – determines what the student is asking.

Knowledge Retrieval – finds relevant institutional information.

Response Generation – produces the final answer.

Response Validation – checks whether the response is supported by the retrieved context.

This modular approach also makes it easier to add new academic tasks in the future.

8.7 Web Application Integration

The complete AI pipeline is integrated into a web-based application. React.js is used to provide the user interface, while Python and FastAPI handle backend communication between the frontend, AI services, and knowledge base.

The overall workflow is:

Student
↓
React.js Interface
↓
FastAPI Backend
↓
Query Processing
↓
RAG + Vector Search
↓
LLM / AI Agent
↓
Context-Aware Response
↓
Student

8.8 Testing and Evaluation

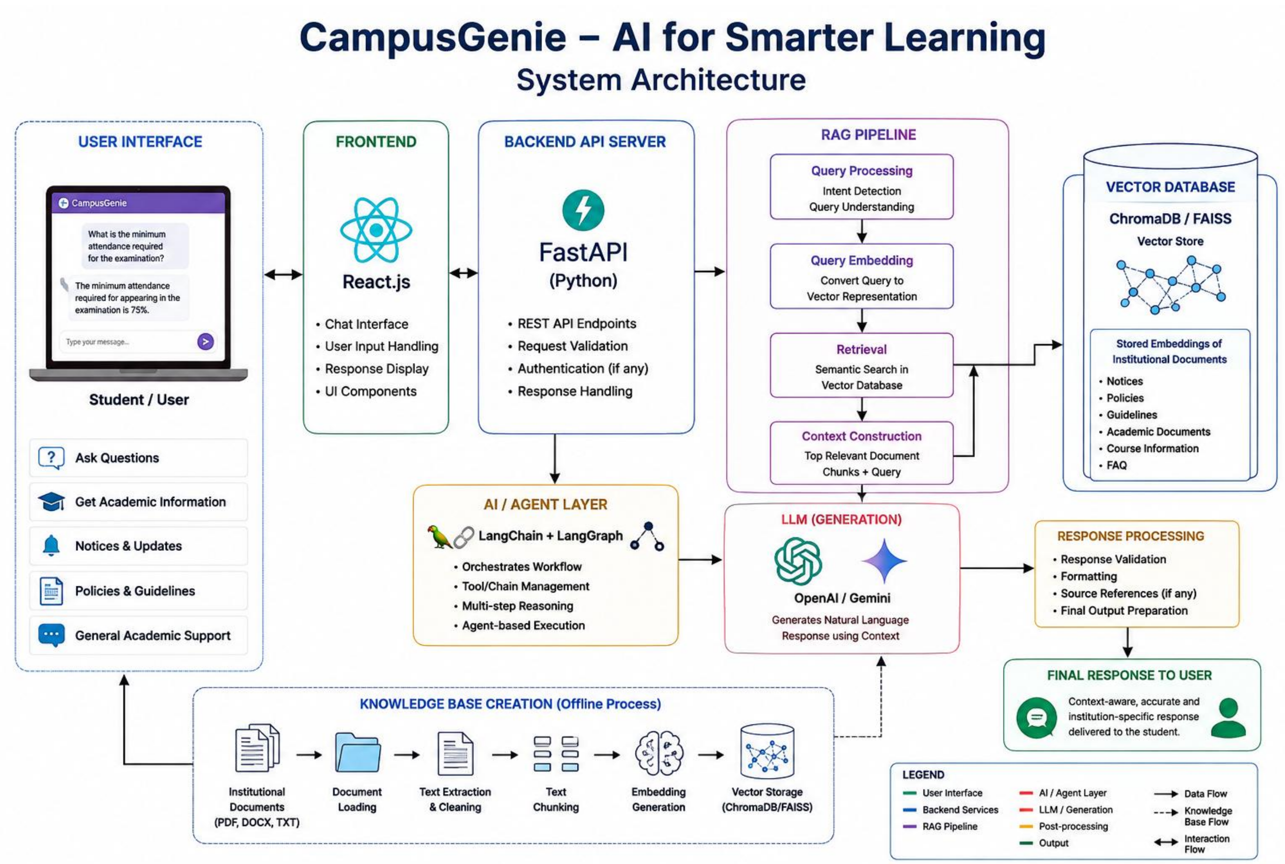
The developed system is tested using different academic queries to evaluate its performance. Testing focuses on retrieval relevance, response accuracy, response time, consistency, and the ability to handle different forms of natural-language questions.

Based on the results, the document-processing, retrieval, prompting, and agent workflows can be refined to improve the overall performance of CampusGenie.

Methodology Summary

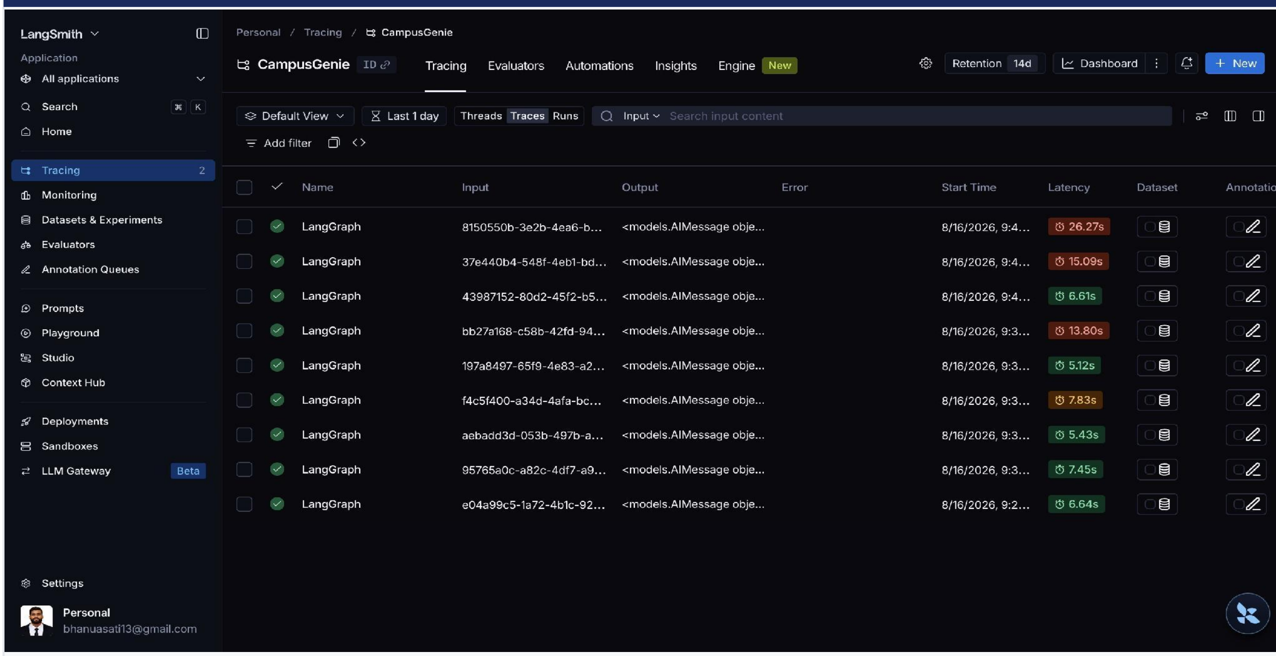
The complete methodology follows:

Document Collection → Pre-processing → Chunking → Embedding Generation → Vector Storage → Query Processing → Semantic Retrieval → RAG → LLM/Agent Processing → Response Generation → Testing and Evaluation.


CampusGenie – AI for Smarter Learning
System Architecture
USER INTERFACE
CampusGenie
What is the minimum attendance required for the examination?
The minimum attendance required for appearing in the examination is 75%.
Type your message...
Student / User
Ask Questions
Get Academic Information
Notices & Updates
Policies & Guidelines
General Academic Support
FRONTEND
React.js
• Chat Interface
• User Input Handling
• Response Display
• UI Components
BACKEND API SERVER
FastAPI
(Python)
• REST API Endpoints
• Request Validation
• Authentication (if any)
• Response Handling
RAG PIPELINE
Query Processing
Intent Detection
Query Understanding
Query Embedding
Convert Query to
Vector Representation
Retrieval
Semantic Search in
Vector Database
Context Construction
Top Relevant Document
Chunks + Query
VECTOR DATABASE
ChromaDB / FAISS
Vector Store
Stored Embeddings of
Institutional Documents
• Notices
• Policies
• Guidelines
• Academic Documents
• Course Information
• FAQ
AI / AGENT LAYER
LangChain + LangGraph
• Orchestrates Workflow
• Tool/Chain Management
• Multi-step Reasoning
• Agent-based Execution
LLM (GENERATION)
OpenAI / Gemini
Generates Natural Language
Response using Context
RESPONSE PROCESSING
• Response Validation
• Formatting
• Source References (if any)
• Final Output Preparation
FINAL RESPONSE TO USER
Context-aware, accurate and institution-specific response delivered to the student.
KNOWLEDGE BASE CREATION (Offline Process)
Institutional Documents (PDF, DOCX, TXT)
Document Loading
Text Extraction & Cleaning
Text Chunking
Embedding Generation
Vector Storage (ChromaDB/FAISS)
LEGEND
User Interface
Backend Services
RAG Pipeline
AI / Agent Layer
LLM / Generation
Post-processing
Output
Data Flow
Knowledge Base Flow
Interaction Flow

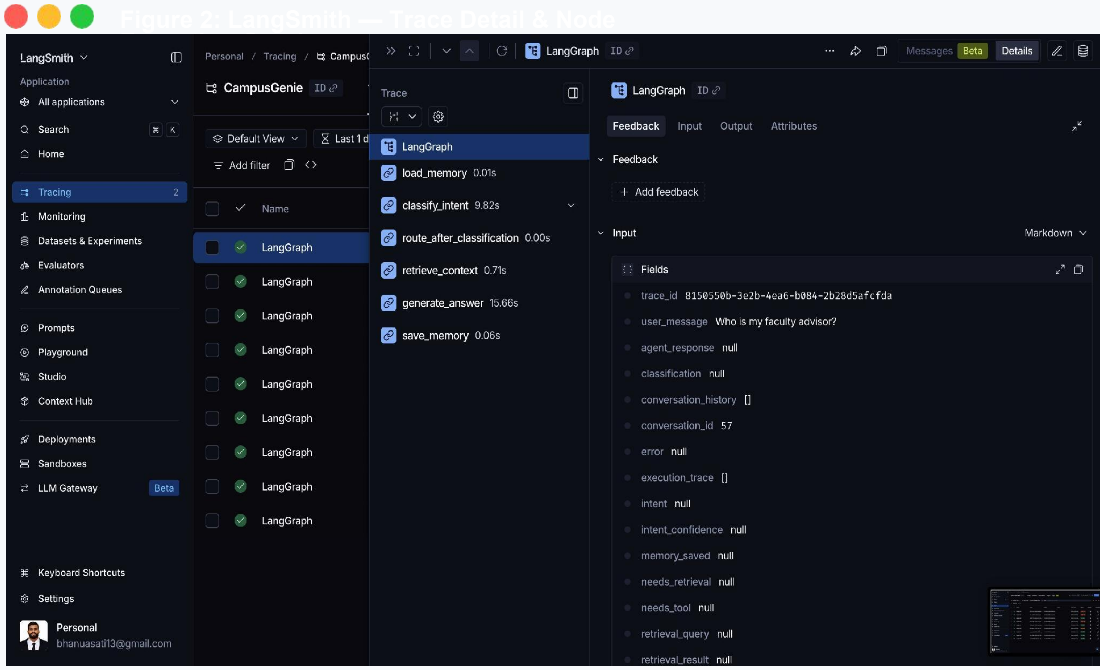
MVP Flow — System Tracing & Observability
LangSmith Tracing Dashboard — CampusGenie LangGraph Agent

Figure 1: LangSmith — CampusGenie Trace List



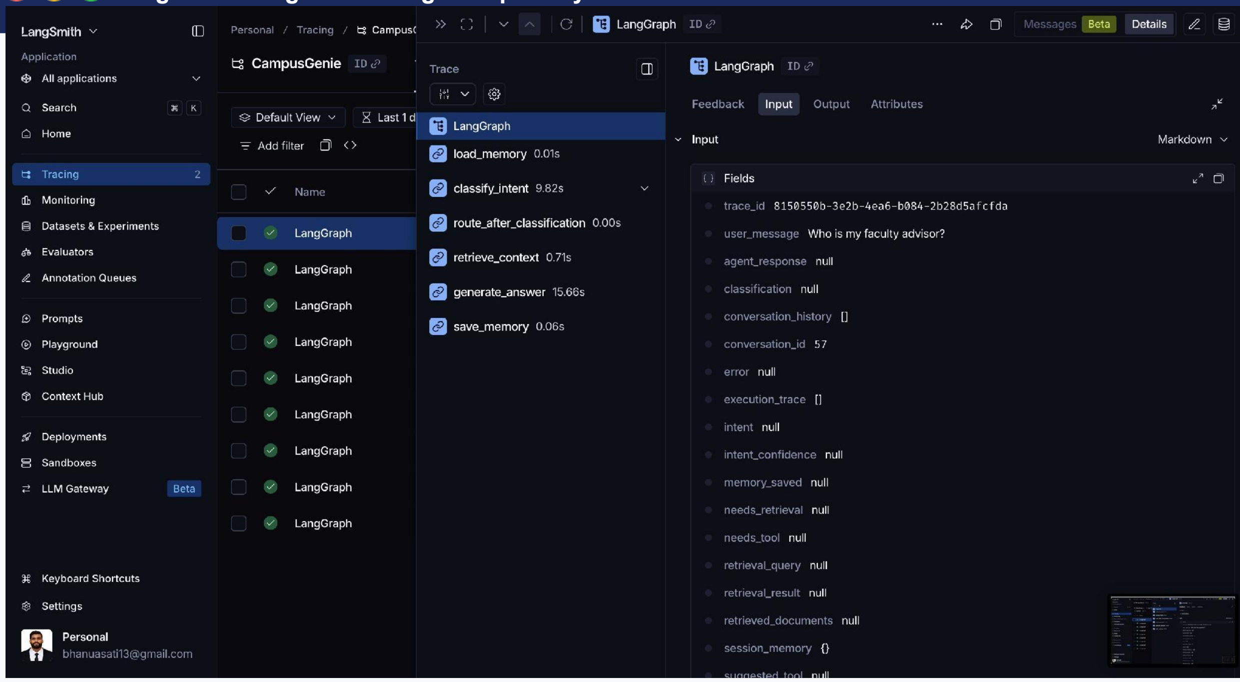
Tracing dashboard showing 9 successful LangGraph runs with latency ranging from 5.12s to 26.27s

Figure 2: LangSmith — Trace Detail & Node



Selected trace showing the full node execution chain with timing: load_memory (0.01s)   classify_intent (9.82s)

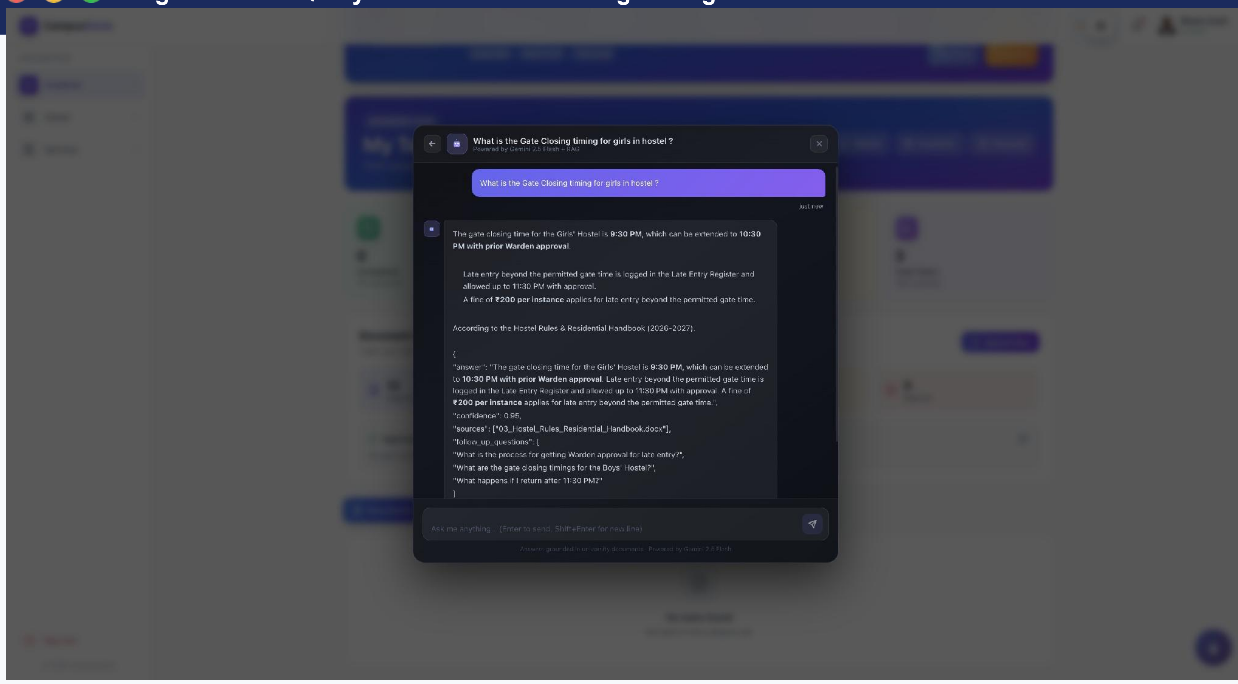
Figure 3: LangSmith — Agent Input Payload



Input fields for trace 8150550b: user_message='Who is my faculty advisor?' with conversation state fields

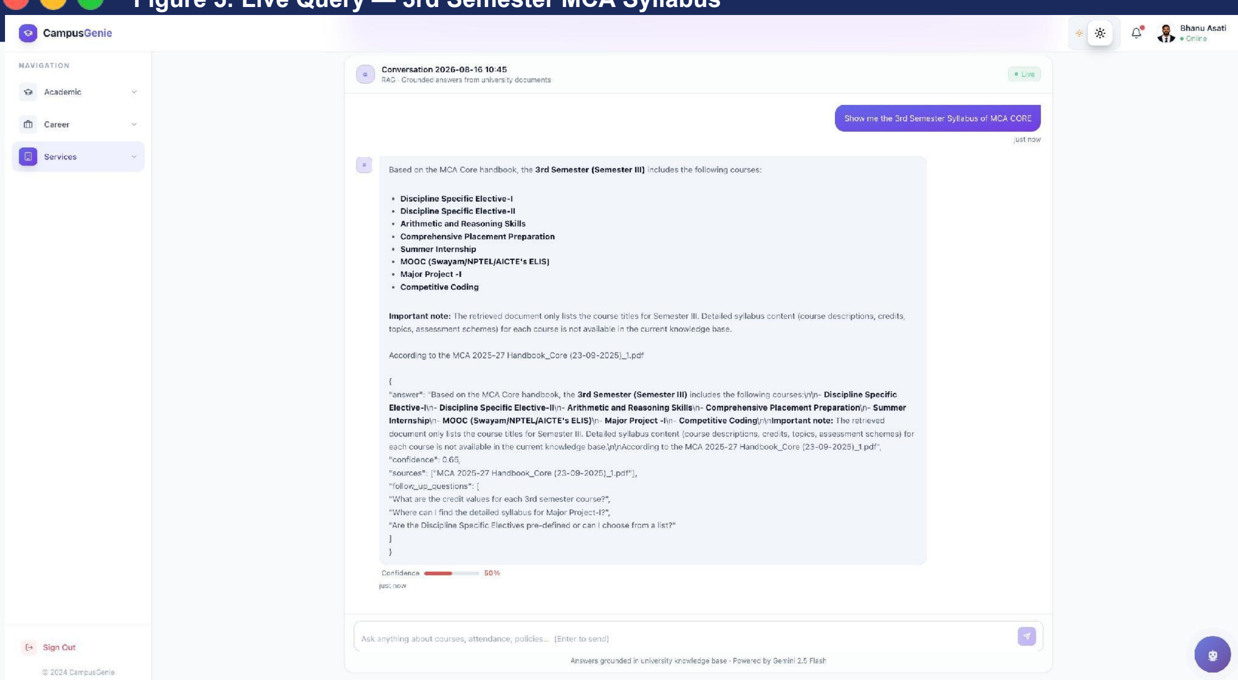
MVP Flow — Live Application Screens
CampusGenie AI Web Interface — Powered by Gemini 2.5 Flash + RAG

Figure 4: Live Query — Hostel Gate Closing Timing



Query: 'What is the Gate Closing timing for girls in hostel?' — Answer grounded in Hostel Rules & Residential Handbook (2026-27), confidence 0.95

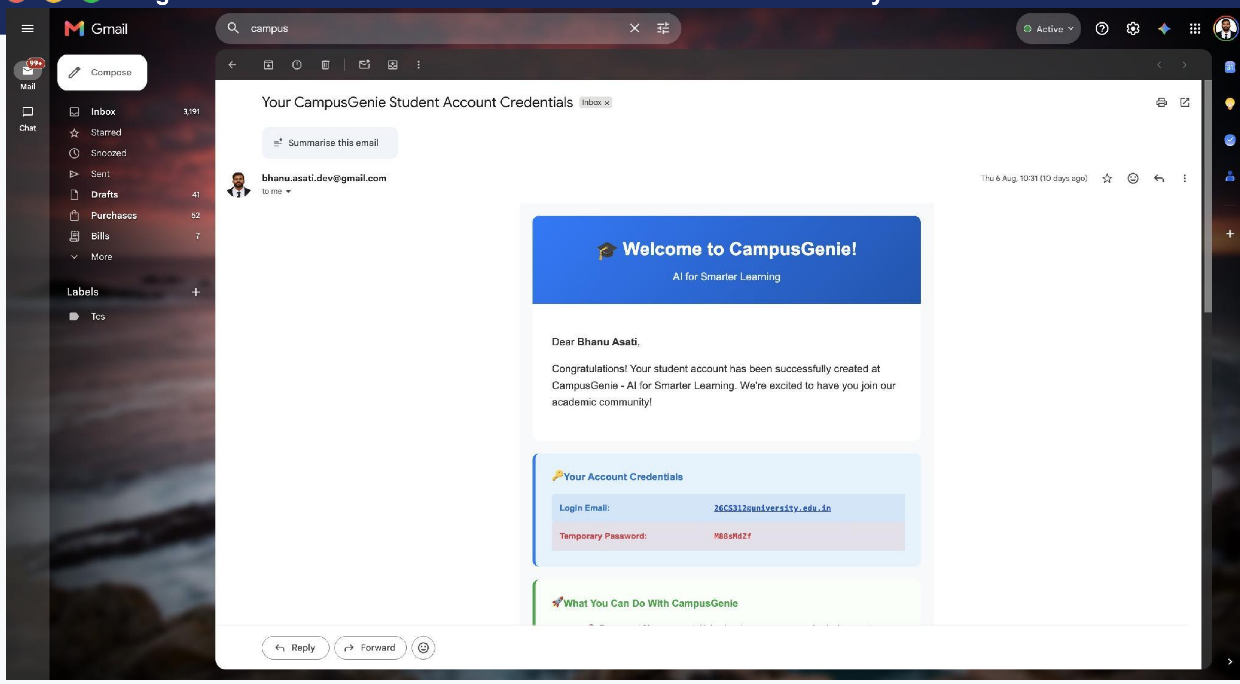
Figure 5: Live Query — 3rd Semester MCA Syllabus



Query: 'Show me the 3rd Semester Syllabus of MCA CORE' — Answer retrieved from MCA 2025-27 Handbook, confidence 50%, with follow-up questions

MVP Flow — Student Onboarding & Email Notification
Automated Welcome Email with University Account Credentials

Figure 6: Student Welcome Email — Account Credentials Delivery



Automated onboarding email: 'Your CampusGenie Student Account Credentials' with university email ID and temporary password

REFERENCES:

[1] C. W. Okonkwo and A. Ade-Ibijola, "Chatbots applications in education: A systematic review," Computers and Education: Artificial Intelligence, vol. 2, Art. no. 100033, 2021, doi: 10.1016/j.caeai.2021.100033.

[2] S. Wollny, J. Schneider, D. Di Mitri, J. Weidlich, M. Rittberger, and H. Drachsler, "Are We There Yet? – A Systematic Literature Review on Chatbots in Education," Frontiers in Artificial Intelligence, vol. 4, Art. no. 654924, 2021, doi: 10.3389/frai.2021.654924.

[3] E. Adamopoulou and L. Moussiades, "Chatbots: History, technology, and applications," Machine Learning with Applications, vol. 2, Art. no. 100006, 2020, doi: 10.1016/j.mlwa.2020.100006.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, 2020.

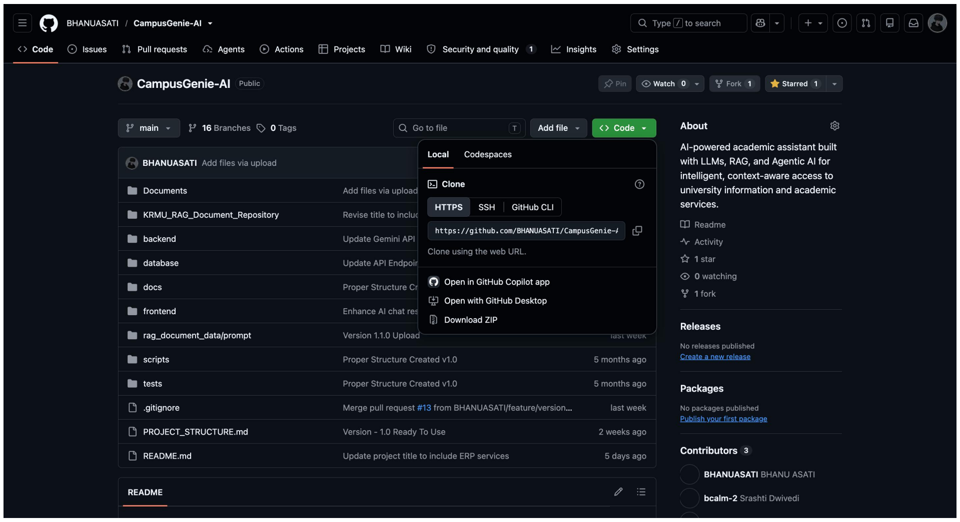
[5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv:2312.10997, 2023.

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv:2210.03629, 2022.

Reference	***Used in Report For***
[1] Okonkwo & Ade-Ibijola (2021)	Educational chatbots, applications, benefits and challenges
[2] Wollny et al. (2021)	Literature review of chatbots in education
[3] Adamopoulou & Moussiades (2020)	Background and evolution of chatbot technology
[4] Lewis et al. (2020)	Foundation of RAG architecture
[5] Gao et al. (2023)	RAG, hallucination, outdated knowledge, and domain-specific information
[6] Yao et al. (2022)	Agentic AI / reasoning + action workflows

Repository Reference

Source Code, Architecture Diagrams & Documentation — GitHub



GitHub Repo Link : https://github.com/BHANUASATI/CampusGenie-AI.git

ThankYou

Team CYC - Control Your Career

Student Name : Bhanu Asati (2501560014) (Leader)

Student Name : Srashti Dwivedi (2501560012)